飞桨

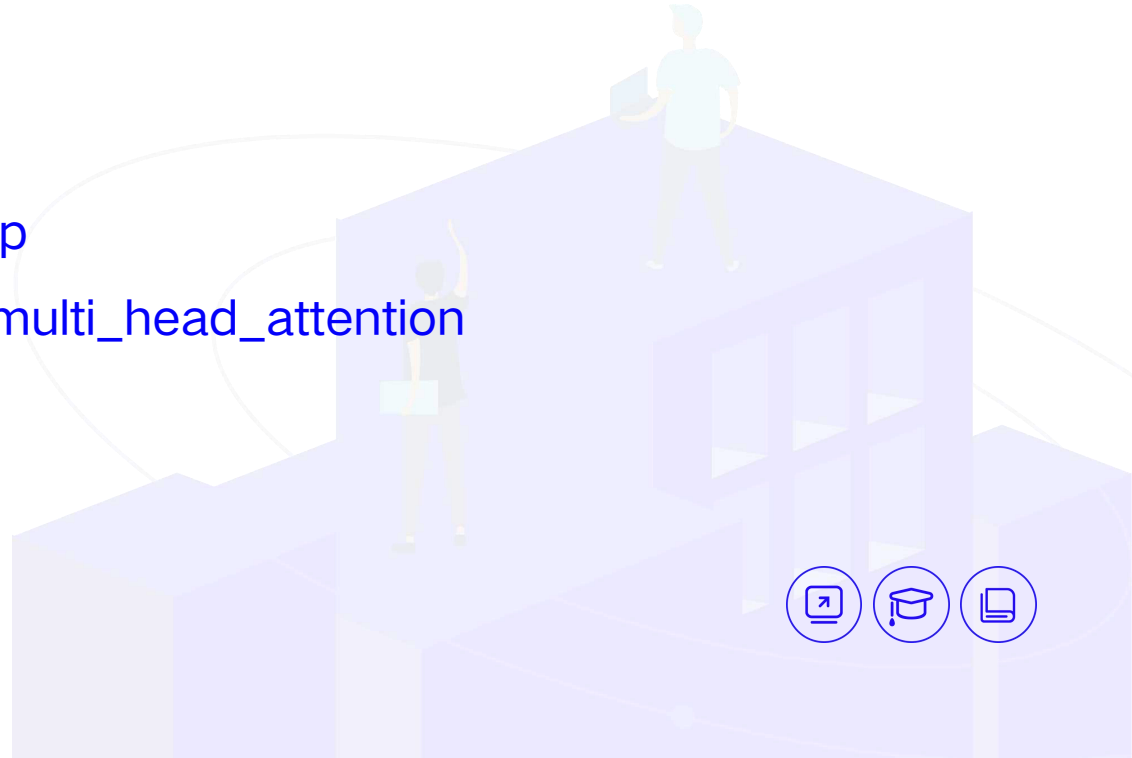
ERNIE生成模型推理部署优化

李振宇

2022-11-25

CONTENT

目录

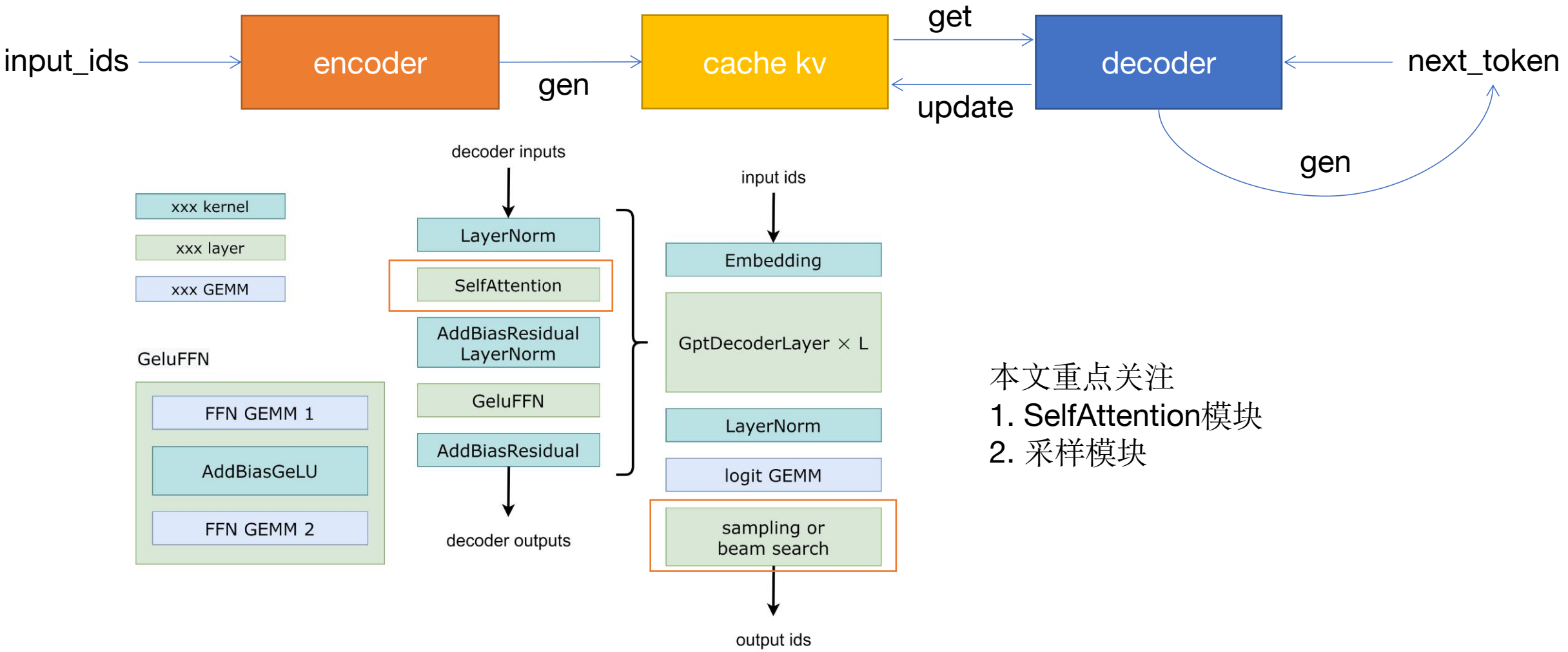


01

ERNIE自然语言生成模型

总体架构

总体架构

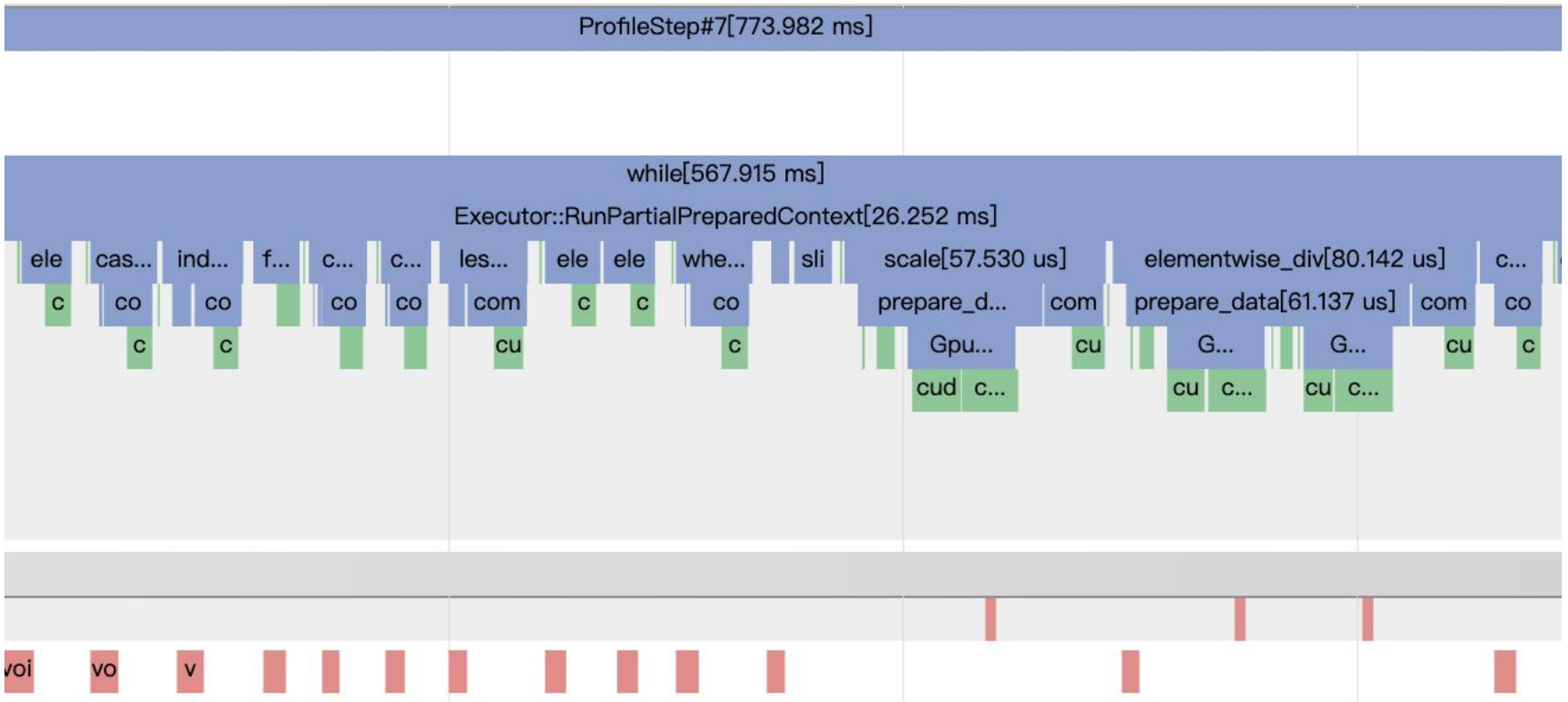


本文重点关注
1. SelfAttention模块
2. 采样模块

02

组网优化

降低同步次数



大多数的同步操作都是由于直接使用CPU上的数据和GPU上的数据作运算，导致产生隐式同步。

这种情况在动转静时特别常见!

减少重复计算

多次执行排序

```
def TopPProcess(probs, top_p, min_tokens_to_keep):
    sorted_probs = paddle.sort(probs, descending=True)
    sorted_indices = paddle.argsort(probs, descending=True)
    cumulative_probs = paddle.cumsum(sorted_probs, axis=-1)
```

多余的softmax操作

```
origin_probs = F.softmax(logits)
origin_probs = paddle.log(origin_probs)
if temperature is not None and temperature != 1.0:
    logits = logits / temperature
probs = F.softmax(logits)
```

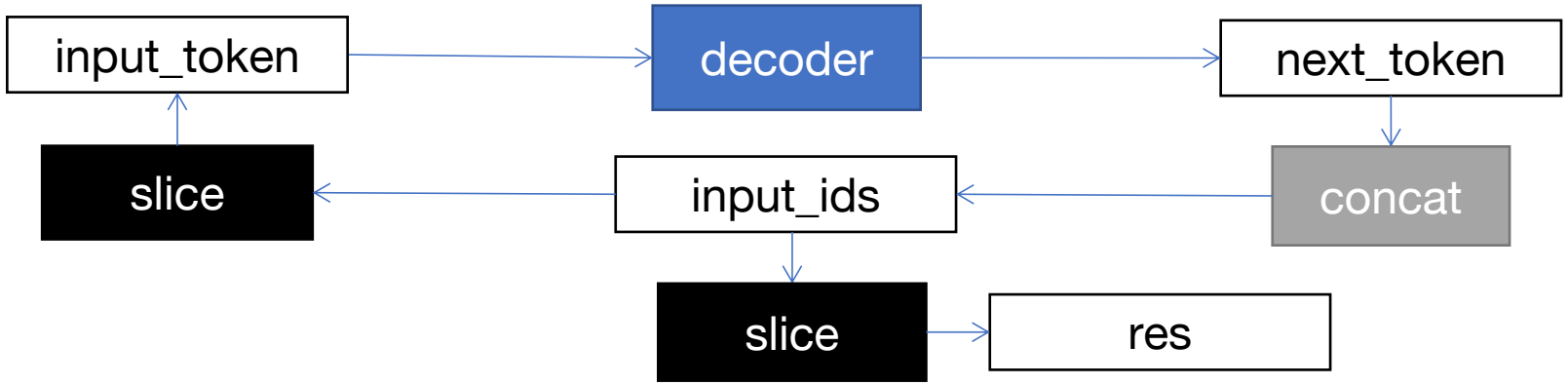
多余的TopK
sampling_ids : [bs, 1]

```
sampling_scores = layers.one_hot(
    layers.unsqueeze(sampling_ids, [1]), probs.shape[1]
)
sampling_scores = sampling_scores * probs - (1 - sampling_scores) * 1e3
topk_scores, topk_indices = layers.topk(
    input=sampling_scores, k=1)
```

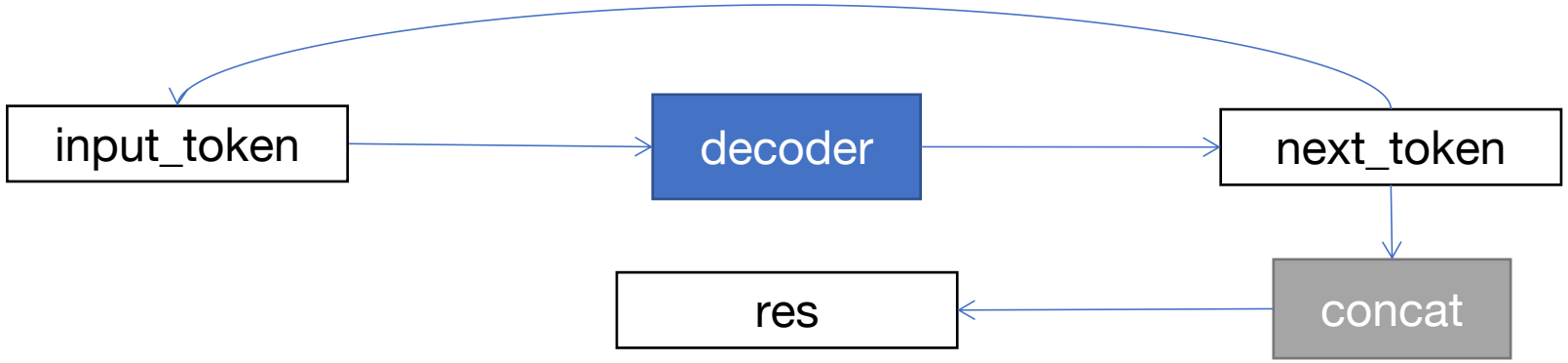
在不改变组网逻辑的前提下，尽可能的减少重复的计算。

优化组网逻辑

FleetX中GPT解码阶段逻辑：



优化后：



03

算子优化

TopK、TopP

1. 原生算子TopK优化。

单TopK算子在vocab_size较大时提速2～4倍。

- BlockReduce->WarpReduce
- 线程块内同步次数：5->3
- 修改补数逻辑
- 优化启动配置
- 自适应MaxLength

shape	k	优化前（ms）	优化后（ms）
[1, 30000]	1	0.0296	0.0101
[1, 300000]	1	0.236	0.0641
[1, 30000]	10	0.102	0.0307
[1, 300000]	10	0.405	0.102
[10, 30000]	1	0.0313	0.0111
[10, 300000]	1	0.251	0.0687
[10, 30000]	10	0.106	0.0346
[10, 300000]	10	0.417	0.104

2. TopP采样融合。

1）常规实现：argsort + cumsum + lt_cond + sampling_id(CPU)

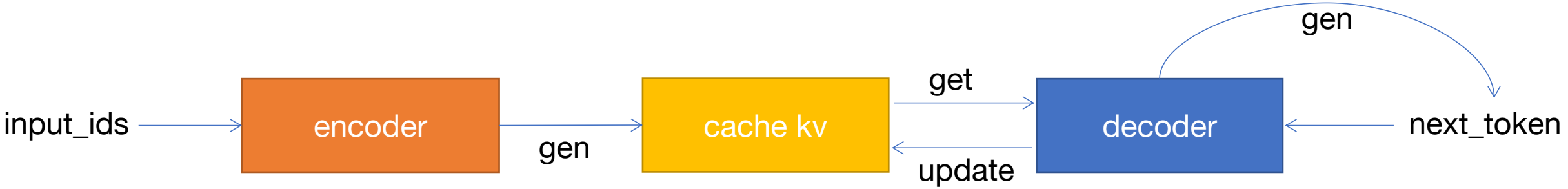
2）融合算子：rand_p + topk + argsort + cumsum(early stop)

NLP自然语言生成模型上最高提速4倍。

3. TopK采样融合。

topk + random(sum(topk))，隐式采样，消除CPU上的Sampling。

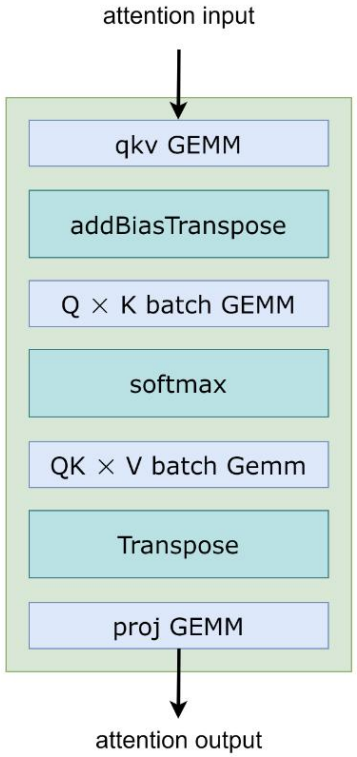
MMHA



encoder和decoder中SelfAttention模块的FMHA。

1. encoder中的FMHA：通用FMHA情况：
   QKV : [3, bs, num_head, seq_len, head_dim]
   Cache_KV : [2, bs, num_head, seq_len, head_dim]
   QK^T : [bs, num_head, seq_len, seq_len]
   Out : [bs, num_head, seq_len, head_dim]

2. decoder中的FMHA，通用FMHA情况：
   QKV : [3, bs, num_head, 1, head_dim]
   Cache_KV : [2, bs, num_head, seq_len, head_dim]
   Concat_KV : [2, bs, num_head, seq_len+1, head_dim]
   QK^T : [bs, num_head, 1, seq_len+1]
   Out : [bs, num_head, 1, head_dim]



MMHA

FusedMultiTransformer中的MMHA对生成模型进行了特殊优化。

1. 提前申请Cache_KV空间: [2, bs, num_head, max_seq_len, dim_head]
   max_seq_len = input_seq_len + max_dec_len

2. encoder阶段通过write_cache_kv将K和V写入Cache_KV中。
   在写入K时进行转制和向量化，写入后Cache_KV中的KV分布如下：
   Cache_K: [bs, num_head, dim_head/x, max_seq_len, x], x = 16 / sizeof(T)
   Cache_V: [bs, num_head, max_seq_len, dim_head]

3. 进行MMHA，内部获取和更新Cache_KV。

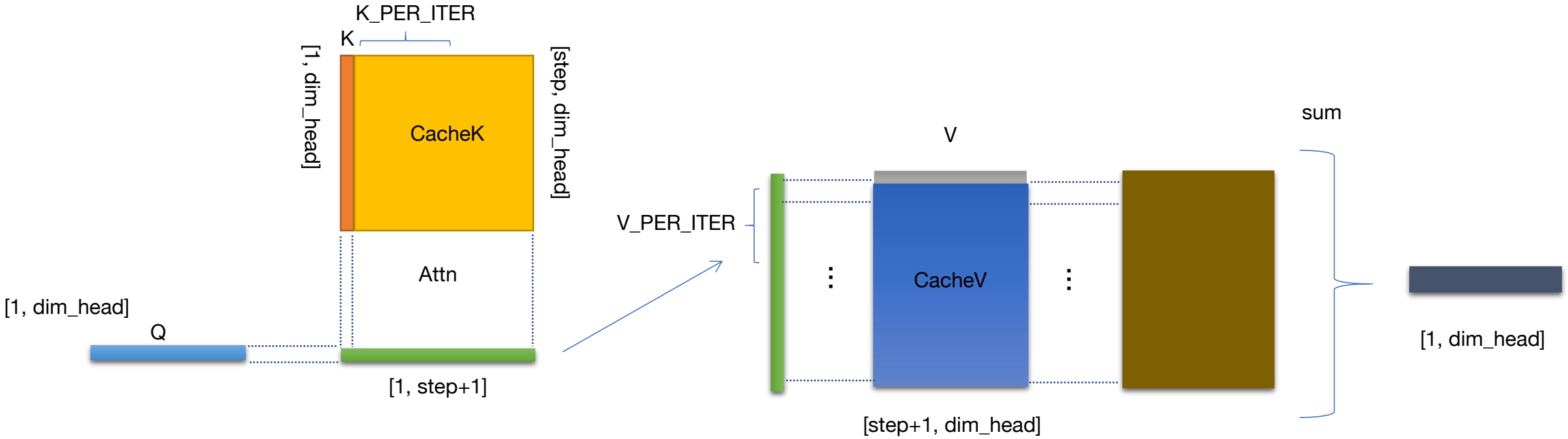
相较于FMHA主要优化思路：消除Concat操作。

MMHA

一些细节：

Q: [bs, num_head, 1, dim_head]
K: [bs, num_head, 1, dim_head]
V: [bs, num_head, 1, dim_head]
Cache_K: [bs, num_head, dim_head/x, max_seq_len, x]
Cache_V: [bs, num_head, max_seq_len, dim_head]

THREADS_PER_KEY: 每个Block中使用多少个线程处理一个timestep的key。
THREADS_PER_VALUE: 每个Block中使用多少个线程处理一个timestep的value。
K_PER_ITER = BLOCKSIZE / THREADS_PER_KEY
V_PER_ITER = BLOCKSIZE / THREADS_PER_VALUE



MMHA

QK矩阵乘的累加部分使用TensorCore

假设BLOCKSIZE=64，FP16，THREADS_PER_KEY = 4， 因此K_PER_ITER=16，K_PER_WARP=8，每个线程处理16个QK的相乘，最终每个线程上保留一个half2。使用mma.m16n8k8指令进行累加。

A

step_id

Row\Col	0 1	2 3	4 5	6 7
0	T0: {a0, a1}	T1: {a0, a1}	T2: {a0, a1}	T3: {a0, a1}
1	T4: {a0, a1}	T5: {a0, a1}	T6: {a0, a1}	T7: {a0, a1}
2				
..				
7	T28: {a0, a1}	T29: {a0, a1}	T30: {a0, a1}	T31: {a0, a1}
8	T0:{a2, a3}	T1:{a2, a3}	T2:{a2, a3}	T3:{a2, a3}
9	T4:{a2, a3}	T5:{a2, a3}	T6:{a2, a3}	T7:{a2, a3}
10				
..				
15	T28:{a2, a3}	T29:{a2, a3}	T30:{a2, a3}	T31:{a2, a3}

全0

B 全1

Row\Column	0	1	2	..	7
0 1	$T0:\{^{b0}_{b1}\}$	$T4:\{^{b0}_{b1}\}$			$T28:\{^{b0}_{b1}\}$
2 3	$T1:\{^{b0}_{b1}\}$	$T5:\{^{b0}_{b1}\}$			$T29:\{^{b0}_{b1}\}$
4 5	$T2:\{^{b0}_{b1}\}$	$T6:\{^{b0}_{b1}\}$			$T30:\{^{b0}_{b1}\}$
6 7	$T3:\{^{b0}_{b1}\}$	$T7:\{^{b0}_{b1}\}$			$T31:\{^{b0}_{b1}\}$

C

Row\Col	0 1	2 3	4 5	6 7
0	T0: {c0, c1}	T1: {c0, c1}	T2: {c0, c1}	T3: {c0, c1}
1	T4: {c0, c1}	T5: {c0, c1}	T6: {c0, c1}	T7: {c0, c1}
2				
..				
7	T28: {c0, c1}	T29: {c0, c1}	T30: {c0, c1}	T31: {c0, c1}
8	T0:{c2, c3}	T1:{c2, c3}	T2:{c2, c3}	T3:{c2, c3}
9	T4:{c2, c3}	T5:{c2, c3}	T6:{c2, c3}	T7:{c2, c3}
10				
..				
15	T28:{c2, c3}	T29:{c2, c3}	T30:{c2, c3}	T31:{c2, c3}

```
hmma_fp32_tensorcore(make_uint2(qk_vec, 0u), 0x3c003c00u).x;
```

04

工作计划

工作计划

1. 持续优化NLP自然语言生成模型性能，支持业务同学部署落地。

2. 探索更加通用的FMHA实现，当FMHA+Concat速度快于MMHA+Write Cache时可以全部替换为通用FMHA。

- FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (https://arxiv.org/pdf/2205.14135.pdf)
- Self-attention Does Not Need O(n^2) Memory (https://arxiv.org/pdf/2112.05682.pdf)

谢谢大家!